GPIB Overview

R2025b

What Is GPIB?

GPIB is a standardized interface that allows you to connect and control multiple devices from various vendors. GPIB is also referred to by its original name HP-IB, or by its IEEE® designation IEEE-488. GPIB functionality has evolved over time, and is described in several specifications:

- The IEEE 488.1-1975 specification defines the electrical and mechanical characteristics of the interface and its basic functional characteristics.
- The IEEE-488.2-1987 specification builds on the IEEE 488.1 specification to define an acceptable minimum configuration and a basic set of instrument commands and common data formats.
- The Standard Commands for Programmable Instrumentation (SCPI) specification builds on the commands given by the IEEE 488.2 specification to define a standard instrument command set that can be used by GPIB or other interfaces.

For many GPIB applications, you can communicate with your instrument without detailed knowledge of how GPIB works. Communication is established through a VISA-GPIB object, which you create using `visadev`.

If your application is straightforward, or if you are already familiar with the GPIB specifications, you can begin with Get Started with GPIB Interface. If you want a high-level description of all the steps you are likely to take when communicating with your instrument, refer to Creating Instrument Objects.

Some of the GPIB functionality is required for all GPIB devices, while other GPIB functionality is optional. Additionally, many devices support only a subset of the SCPI command set, or use a different vendor-specific command set. Refer to your device documentation for a complete list of its GPIB capabilities and its command set.

Important GPIB Features

The important GPIB features are described in the following sections. For detailed information about GPIB functionality, see the appropriate references in the Bibliography.

Bus and Connector

The GPIB bus is a cable with two 24-pin connectors that allow you to connect multiple devices to each other. The bus and connector have these features and limitations:

- You can connect up to 15 devices to a bus.
- You can connect devices in a star configuration, a linear configuration, or a combination of configurations.
- To achieve maximum data transfer rates, the cable length should not exceed 20 meters total or an average of 2 meters per device. You can eliminate these restrictions by using a bus extender.

GPIB Devices

Each GPIB device must be some combination of a *Talker*, a *Listener*, or a *Controller*. A Controller is typically a board that you install in your computer. Talkers and Listeners are typically instruments such as oscilloscopes, function generators, multimeters, and so on. Most modern instruments are both Talkers and Listeners.

- Talkers — A Talker transmits data over the interface when addressed to talk by the Controller. A GPIB system can contain only one Talker at a time.
- Listeners — A Listener receives data over the interface when addressed to listen by the Controller. A GPIB system can contain up to 14 Listeners at a given time. Typically, the Controller is a Talker while one or more instruments on the GPIB are Listeners.
- Controllers — The Controller specifies which devices are Talkers or Listeners. A GPIB system can contain multiple Controllers. One of them is designated the System Controller. However, only one Controller can be active at a given time. The current active controller is the Controller-In-Charge (CIC). The CIC can pass control to an idle Controller, but only the System Controller can make itself the CIC.

When the Controller is not sending messages, then a Talker can send messages. Typically, the CIC is a Listener while another device is enabled as a Talker.

Each Controller is identified by a unique board index number. Each Talker/Listener is identified by a unique primary address ranging from 0 to 30, and by an optional secondary address, which can be 0 or can range from 96 to 126.

GPIB Data

Two types of data can be transferred over GPIB, *instrument data* and *interface messages*:

- Instrument data — Instrument data consists of vendor-specific commands that configure your instrument, return measurement results, and so on. For a complete list of commands supported by your instrument, refer to its documentation.
- Interface messages — Interface messages are defined by the GPIB standard and consist of commands that clear the GPIB bus, address devices, return self-test results, and so on.

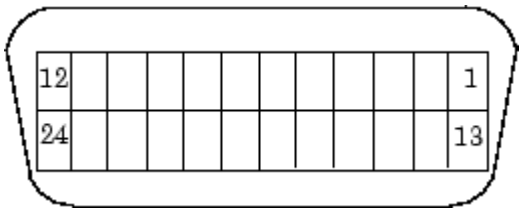
Data transfer consists of one byte (8 bits) sent in parallel. The data transfer rate across the interface is limited to 1 megabyte per second. However, this data rate is usually not achieved in practice, and is limited by the slowest device on the bus.

GPIB Lines

GPIB consists of 24 lines, which are shared by all instruments connected to the bus. 16 lines are used for signals, while eight lines are for ground. The signal lines are divided into these groups:

- Eight data lines
- Five interface management lines
- Three handshake lines

The signal lines use a low-true (negative) logic convention with TTL levels. This convention means that a line is low (true or asserted) when it is a TTL low level, and a line is high (false or unasserted) when it is a TTL high level. The pin assignment scheme for a GPIB connector is shown in the following figure and table.



GPIB Pin and Signal Assignments

Pin	Label	Signal Name	Pin	Label	Signal Name
1	DIO1	Data transfer	13	DIO5	Data transfer
2	DIO2	Data transfer	14	DIO6	Data transfer
3	DIO3	Data transfer	15	DIO7	Data transfer
4	DIO4	Data transfer	16	DIO8	Data transfer
5	EOI	End Or Identify	17	REN	Remote Enable
6	DAV	Data Valid	18	GND	DAV ground
7	NRFD	Not Ready For Data	19	GND	NRFD ground
8	NDAC	Not Data Accepted	20	GND	NDAC ground
9	IFC	Interface Clear	21	GND	IFC ground
10	SRQ	Service Request	22	GND	SRQ ground
11	ATN	Attention	23	GND	ATN ground

Pin	Label	Signal Name	Pin	Label	Signal Name
12	Shield	Chassis ground	24	GND	Signal ground

Data Lines

The eight data lines, DIO1 through DIO8, are used for transferring data one byte at a time. DIO1 is the least significant bit, while DIO8 is the most significant bit. The transferred data can be an instrument command or a GPIB interface command.

Data formats are vendor-specific and can be text-based (ASCII) or binary. GPIB interface commands are defined by the IEEE 488 standard.

Interface Management Lines

The interface management lines control the flow of data across the GPIB interface.

GPIB Interface Management Lines

Line	Description
ATN	Used by the Controller to inform all devices on the GPIB that bytes are being sent. If the ATN line is high, the bytes are interpreted as an instrument command. If the ATN line is low, the bytes are interpreted as an interface message.
IFC	Used by the Controller to initialize the bus. If the IFC line is low, the Talker and Listeners are unaddressed, and the System Controller becomes the Controller-In-Charge.
REN	Used by the Controller to place instruments in remote or local program mode. If REN is low, all Listeners are placed in remote mode, and you cannot change their settings from the front panel. If REN is high, all Listeners are placed in local mode.
SRQ	Used by Talkers to asynchronously request service from the Controller. If SRQ is low, then one or more Talkers require service (for example, an error such as invalid command was received). You issue a serial poll to determine which Talker requested service. The poll automatically sets the SRQ line high.
EOI	If the ATN line is high, the EOI line is used by Talkers to identify the end of a byte stream such as an instrument command. If the ATN line is low, the EOI line is used by the Controller to perform a parallel poll (not supported by the toolbox).

Handshake Lines

The three handshake lines, DAV, NRFD, and NDAC, are used to transfer bytes over the data lines from the Talker to one or more addressed Listeners.

Before data is transferred, all three lines must be in the proper state. The active Talker controls the DAV line and the Listener(s) control the NRFD and NDAC lines. The handshake process allows for error-free data transmission.

Handshake Lines

Line	Description
DAV	Used by the Talker to indicate that a byte can be read by the Listeners.
NRFD	Indicates whether the Listener is ready to receive the byte.
NDAC	Indicates whether the Listener has accepted the byte.

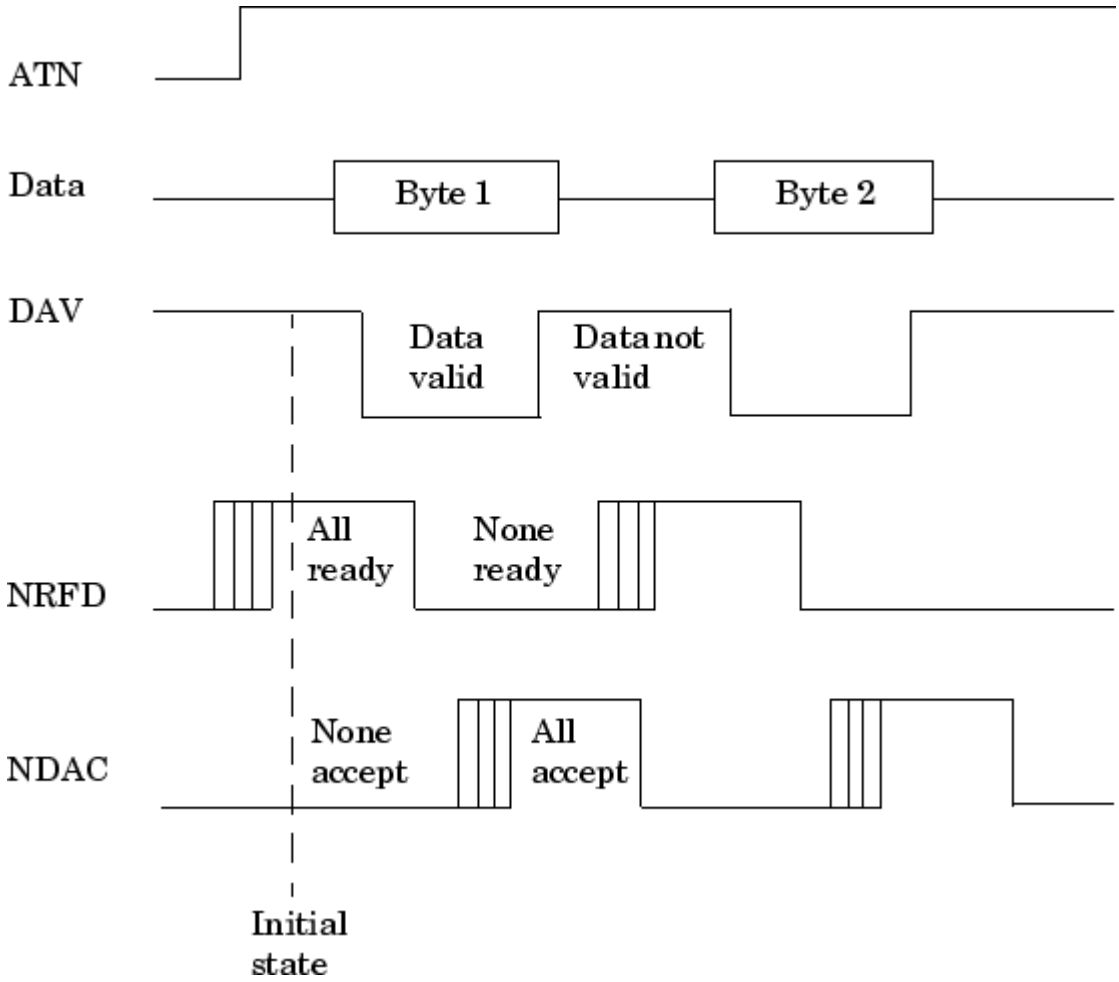
The handshaking process follows these steps:

1. Initially, the Talker holds the DAV line high indicating no data is available, while the Listeners hold the NRFD line high and the NDAC line low indicating they are ready for data and no data is accepted, respectively.

2. When the Talker puts data on the bus, it sets the DAV line low, which indicates that the data is valid.
3. The Listeners set the NRFD line low, which indicates that they are not ready to accept new data.
4. The Listeners set the NDAC line high, which indicates that the data is accepted.
5. When all Listeners indicate that they have accepted the data, the Talker sets the DAV line high indicating that the data is no longer valid. The next byte of data can now be transmitted.
6. The Listeners hold the NRFD line high indicating they are ready to receive data again, and the NDAC line is held low indicating no data is accepted.

> **Note**
>
> If the ATN line is high during the handshaking process, the information is considered data such as an instrument command. If the ATN line is low, the information is considered a GPIB interface message.

The handshaking steps are shown in the following figure.

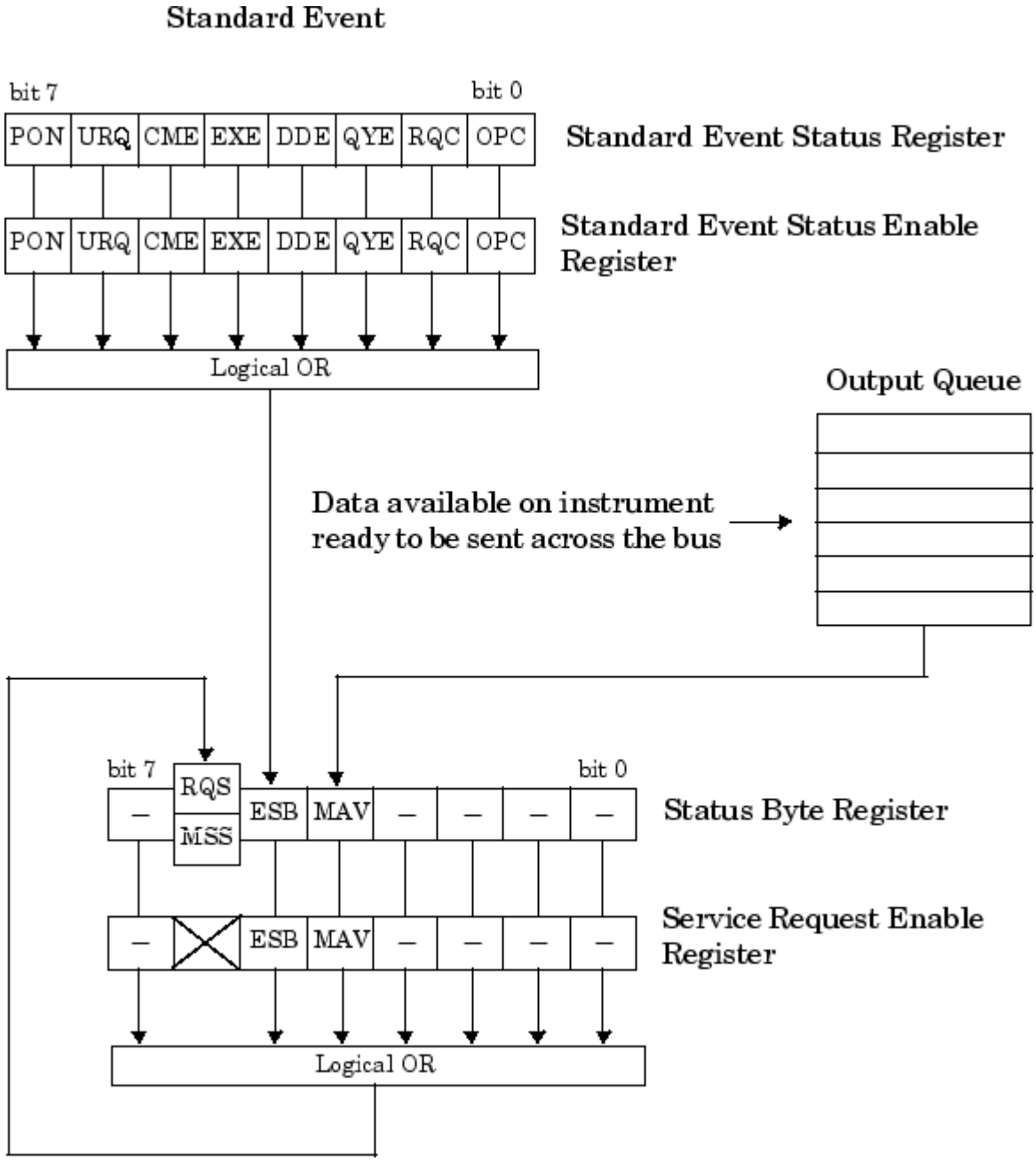


Status and Event Reporting

GPIB provides a system for reporting status and event information. With this system, you can find out if your instrument has data to return, whether a command error occurred, and so on. For many instruments, the reporting system consists of four 8-bit registers and two queues (output and event). The four registers are grouped into these two functional categories:

- Status Registers — The Status Byte Register (SBR) and Standard Event Status Register (SESR) contain information about the state of the instrument.
- Enable Registers — The Event Status Enable Register (ESER) and the Service Request Enable Register (SRER) determine which types of events are reported to the status registers and the event queue. ESER enables SESR, while SRER enables SBR.

The status registers, enable registers, and output queue are shown in the following figure.



Status Byte Register

Each bit in the Status Byte Register (SBR) is associated with a specific type of event. When an event occurs, the instrument sets the appropriate bit to 1. You can enable or disable the SBR bits with the Service Request Enable Register (SRER). You can determine which events occurred by reading the enabled SBR bits.

Status Byte Register Bits

Bit	Label	Description
0-3	–	Instrument-specific summary messages.
4	MAV	The Message Available bit indicates if data is available in the Output Queue. MAV is 1 if the Output Queue contains data. MAV is 0 if the Output Queue is empty.
5	ESB	The Event Status bit indicates if one or more enabled events have occurred. ESB is 1 if an enabled event occurs. ESB is 0 if no enabled events occur. You enable events with the Standard Event Status Enable Register.
6	MSS	The Master Summary Status summarizes the ESB and MAV bits. MSS is 1 if either MAV or ESB is 1. MSS is 0 if both MAV and ESB are 0. This bit is obtained from the ∗STB? command.

Bit	Label	Description
	RQS	The Request Service bit indicates that the instrument requests service from the GPIB controller. This bit is obtained from a serial poll.
7	–	Instrument-specific summary message.

For example, if you want to know when a specific type of instrument error occurs, enable bit 5 of the SRER. Additionally, enable the appropriate bit of the Standard Event Status Enable Register so that the error event of interest is reported by the ESB bit of the SBR.

Standard Event Status Register

Each bit in the Standard Event Status Register (SESR) is associated with a specific state of the instrument. When the state changes, the instrument sets the appropriate bits to 1. You can enable or disable the SESR bits with the Standard Event Status Enable Register (ESER). You can determine the state of the instrument by reading the enabled SESR bits. The SESR bits have the following descriptions.

SESR Bits

Bit	Label	Description
0	OPC	The Operation Complete bit indicates that all commands have completed.
1	RQC	The Request Control bit is not used by most instruments.
2	QYE	The Query Error bit indicates that the instrument attempted to read an empty output buffer, or that data in the output buffer was lost.
3	DDE	The Device Dependent Error bit indicates that a device error occurred (such as a self-test error).
4	EXE	The Execution Error bit indicates that an error occurred when the device was executing a command or query.
5	CME	The Command Error bit indicates that a command syntax error occurred.
6	URQ	The User Request bit is not used by most instruments.
7	PON	The Power On bit indicates that the device is powered on.

For example, if you want to know when an execution error occurs, enable bit 4 of the ESER. Additionally, enable bit 5 of the SRER so that the error event of interest is reported by the ESB bit of the SBR.

Reading and Writing Register Information

This section describes the common GPIB commands used to read and write status and event register information.

Register Commands

Register	Operation	Command	Description
SESR	Read	`*ESR?`	Return a decimal value that corresponds to the weighted sum of all the bits set in the SESR register.
	Write	N/A	You cannot write to the SESR register.
ESER	Read	`*ESE?`	Return a decimal value that corresponds to the weighted sum of all the bits enabled by the `*ESE` command.

Register	Operation	Command	Description
	Write	∗ESE	Write a decimal value that corresponds to the weighted sum of all the bits you want to enable in the SESR register.
SBR	Read	∗STB?	Return a decimal value that corresponds to the weighted sum of all the bits set in the SBR register. This command returns the same result as a serial poll except that the MSS bit is not cleared.
	Write	N/A	You cannot write to the SBR register.
SRER	Read	∗SRE?	Return a decimal value that corresponds to the weighted sum of all the bits enabled by the ∗SRE command.
	Write	∗SRE	Write a decimal value that corresponds to the weighted sum of all the bits you want to enable in the SBR register.

For example, to enable bit 4 of the SESR, you write the command ∗ESE 16. To enable bit 4 and bit 5 of the SESR, you write the command ∗ESE 48. To enable bit 5 of the SBR, you write the command ∗SRE 32.

To see how to use many of these commands in the context of an instrument control session, refer to Execute Serial Poll.

See Also

`visadev`

Topics

Get Started with GPIB Interface

Write and Read GPIB Data

Send Trigger to Instrument

Execute Serial Polls

External Websites

GPIB Instruments and MATLAB